WHITE NOISE SAMPLING ON GRAPHS

DIYA PATEL AND PANAYOT S. VASSILEVSKI

Abstract. This project extends by analogy a known Monte Carlo sampling procedure for discretized PDEs (partial differential equations) now to graph Laplacian problems. We implement a white noise sampling framework on graphs that generates spatially correlated random permeability fields on graph edges, solves a Darcy flow problem on the resulting weighted graph, and extracts a quantity of interest (QoI) that characterizes flow through the network. The framework is demonstrated on the Rhesus Macaque Brain Connectivity Network, where the QoI measures total neural signal flux from peripheral sensory regions to central processing hubs under uncertain synaptic connection strengths. We show that the Monte Carlo estimator converges reliably and that the resulting QoI distribution is approximately normal, consistent with theoretical predictions.

1. Problem formulation

We are given an undirected connected graph G with vertices $V = \{1, 2, \ldots n\}$ and a set of edges $E = \{e = (i, j)\} \subset V \times V$. The graph is uniquely described by its adjacency matrix $A = (a_{ij})$ with nonzero entries $a_{i,j} = a_{j,i} = 1$ if the vertices i and j form an edge $e = (i, j)$.

Let d_i be the degree of vertex i and $D = \mathrm{diag}(d_i)$ be the diagonal degree matrix. We also need the graph Laplacian matrix $L = D - A$, which can be represented in terms of the vectors $\mathbf{d}_e \in \mathbb{R}^n$ for each edge $e = (i, j)$ with entries 1 and -1 at positions i and j respectively. Namely, we have $L = \sum_e \mathbf{d}_e \mathbf{d}_e^T$. We will be also interested in the weighted graph Laplacian for given edge-based weights $k = (k_e)_{e \in E}$

$$\sum_{e \in E} k_e \mathbf{d}_e \mathbf{d}_e^T. \tag{1.1}$$

The graph Laplacian L plays a central role in this project. For any vector $\mathbf{v} \in \mathbb{R}^n$, the quadratic form of L satisfies:

$$\mathbf{v}^T L \mathbf{v} = \sum_{e=(i,j) \in E} (v_i - v_j)^2. \tag{1.2}$$

Date: May 29, 2026–beginning; Today is October 18, 2026.

2020 *Mathematics Subject Classification.* 65F10, 65N20, 65N30.

Key words and phrases. fractional order hierarchical decompositions, multigrid, fractional order white noise sampling.

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research Program. The author benefited also from the Research Training Group (RTG) activities under NSF grant DMS-2136228.

This identity shows that L measures the total squared variation of $\mathbf{v}$ across all edges. When L appears in a linear system, the solution is forced to vary smoothly across connected vertices – a property that underpins the spatial correlation structure generated in our sampling procedure.

First, we compute the minimal positive eigenvalue $\lambda^+_{\min}$ of $D^{-1}L$. For a connected graph, we have

$$\lambda^+_{\min} = \min_{\mathbf{v}\perp\mathbf{1}=(1)} \frac{\mathbf{v}^T L\mathbf{v}}{\mathbf{v}^T D\mathbf{v}}.$$

This Rayleigh quotient characterization reveals the meaning of $\lambda^+_{\min}$: it is the minimum ratio of variation to connectivity across all non-constant vectors on the graph. The denominator $\mathbf{v}^T D\mathbf{v}$ normalizes by degree, ensuring that high-degree vertices do not artificially dominate. Intuitively, $\lambda^+_{\min}$ is the graph's fundamental frequency - the slowest non-trivial mode of variation consistent with the graph's connectivity structure. A small $\lambda^+_{\min}$ indicates a bottleneck in the graph; a large value indicates robust connectivity throughout.

Then, we form the shifted graph Laplacian $\mathcal{L}_\sigma = L+\sigma^2\lambda^+_{\min}D$ for a given parameter σ^2 (e.g., $\sigma^2 = 0.25$). We note that $\mathcal{L}_\sigma$ is s.p.d.(symmetric positive definite).

Our sampling procedure is as follows:

- We define a graph white noise $\mathbf{w} = (w_e)_{e\in E}$ where w_e are $\mathcal{N}(0,1)$ random numbers associated with each edge e.
- Then we form the vector $\mathbf{f} = (f_i)_{i=1}^n$ where f_i is defined for every graph vertex i as follows
$$f_i = \sum_{e=(i,j)} w_e$$
- The sampling process takes a random edge-based vector $\mathbf{w} = (w_e)$, forms the vertex-based vector $\mathbf{f}$ and computes the solution $\mathbf{u}$ of the system of equations

$$\mathcal{L}_\sigma\mathbf{u} = \lambda^+_{\min}\mathbf{f}. \tag{1.3}$$

The sample $\mathbf{u} = (u_i)$ corresponding to a $\mathbf{w}$ is used as a random edge-based coefficient $k = (k_e)$, where

$$k_e = \exp\left(\frac{u_i + u_j}{2}\right),\ e = (i,j). \tag{1.4}$$

In a Monte Carlo method, we will solve at every Monte Carlo step a weighted graph Laplacian problem with the random edge-based coefficient k

$$\sum_e k_e\mathbf{d}_e\mathbf{d}_e^T\mathbf{p} = 0, \tag{1.5}$$

subject to some values of $\mathbf{p}$ prescribed on a given portion of vertices Γ_{in} of the graph. We then compute a respective quantity of interest (QoI), $Q(\mathbf{p})$, which depend on the solution $\mathbf{p}$. More specifically, the QoI, $Q(\mathbf{p})$, is typically a function of a vector of entries of $\mathbf{p}$ corresponding to another portion of graph vertices Γ_{out} that are of particular interest. That is, we are given two vertex sets of interest, an input one, Γ_{in}, and an output one, Γ_{out}. We are interested how the input data propagates into the quantity of interest through the output values of the solution $\mathbf{p}$ on Γ_{out}.

2. Connection to the PDE Setting

This project extends by analogy a well-established procedure for sampling spatially correlated Gaussian fields in the continuous PDE setting. In that setting, one generates a sample by solving a fractional-order elliptic PDE of the form:

$$(-\Delta + \kappa^2)^{\alpha/2} u = \mathcal{W}, \tag{2.1}$$

where Δ is the continuous Laplace operator, $\kappa > 0$ controls the correlation length, $\alpha > 0$ controls the smoothness of the resulting field, and $\mathcal{W}$ is spatial white noise. The solution u is a Gaussian random field whose covariance function is a Matérn kernel, characterized by exponential decay with distance. ***Put a citation here.***

The graph analogue replaces each component of this procedure as follows. The continuous Laplacian $-\Delta$ is replaced by the graph Laplacian L. The shift κ^2 is replaced by $\sigma^2 \lambda_{\min}^+$, calibrated to the graph's own spectral scale. The continuous spatial white noise $\mathcal{W}$ is replaced by graph white noise $w_e \sim \mathcal{N}(0, 1)$ assigned independently to each edge. The fractional power $\alpha/2$ is set to 1 in the current implementation, with extensions to fractional powers constituting future work.

The key insight is that solving $\mathcal{L}_\sigma \mathbf{u} = \sqrt{\lambda_{\min}}\, \mathbf{f}$ on a graph produces a field with the same qualitative properties as the PDE solution: Gaussian marginals, spatial correlation that decays with graph distance, and a covariance structure tied to the graph's spectral properties. Specifically, since $\mathbf{f} = B\mathbf{w}$ is a linear transformation of the Gaussian vector $\mathbf{w}$, it is itself Gaussian. The solution $\mathbf{u} = \mathcal{L}_\sigma^{-1} \lambda_{\min}^+ \mathbf{f}$ is then also Gaussian, with covariance matrix:

$$\mathrm{Cov}(\mathbf{u}) = (\lambda_{\min}^+)^2 \mathcal{L}_\sigma^{-1} B B^T \mathcal{L}_\sigma^{-T}. \tag{2.2}$$

The covariance between two vertices i and j in $\mathbf{u}$ depends on their graph-theoretic distance, decaying as vertices become more separated in the graph — the discrete analogue of the Matérn correlation structure.

3. Needed tools

The project needs efficient graph Laplacian solvers, one for the shifted graph Laplacian problem (**??**) with white noise r.h.s. and another one, (**??**), for the weighted graph Laplacian with the computed random coefficient (**??**). The two solvers have fundamentally different computational characteristics. Since $\mathcal{L}_\sigma$ is fixed across all Monte Carlo steps, its Cholesky factorization can be computed once and reused at every step. In contrast, L_k changes at every step because k changes, requiring a fresh solve each time. This asymmetry motivates different solver strategies for the two systems, as described in Section **??**.

4. Implementation

4.1. **Phase 0: Data Import and Visualization.** Phase 0 essentially consists of two crucial parts: importing and visualizing network data.

4.1.1. *Importing the Data.* In this project, we are primarily dealing with unweighted and undirected graphs.

Data is imported by calling the `load_graph()` function, which takes a filepath as input and returns an edge list, node count, and a dictionary of edge weights if present. The function handles two most common sources of network data files: Matrix Market (`.mtx`) files, parsed directly using `scipy.io.mmread`, and plain edge list text files, whose delimiter (whitespace or comma) and node indexing convention (0- or 1-indexed) is auto-detected from the file's content. Self-loops are discarded, and duplicate edges are removed (since we are dealing with undirected graphs). ***maybe rephrase as, "since we are treating all graphs as undirected at this stage"?***

Determining the total number of nodes is a crucial step here. One approach is to use the largest node index as an indication of the total number of node. However, this approach produces inaccurate results in many real-world datasets where non-contiguous node identifiers are used. For instance, during our research, we came across a graph that had node indices from 25 through 29, representing a 5-node graph. However, if we were to use the highest-node index approach here, we would end up falsely concluding that the graph has 30 nodes. In light of this, we used a relatively more robust approach to compute the total number of nodes in the graph in which we count the number of distinct nodes actually referenced across all edges to return the node count. Moreover, after collecting the full set of referenced node IDs, we re-map them to a contiguous range starting at 0, and report the true count of distinct nodes as n.

`load_graph()` returns a dictionary consisting of edge weights in case of weighted graphs as one of its outputs even though this project specifically deals with unweighted graphs to avoid data loss. The inherent edge weights of a weighted graph are discarded by running `edge_weights = None` before proceeding further with this pipeline, so that the edge permeabilities are derived entirely from the white noise sampling procedure described in Phases 3 and 4 of the pipeline and do not have any inherent bias to begin with.

4.1.2. *Visualizing the Data.* There is an optional but recommended step of visualizing the graph to get a visual sense of how the vertices and edges are laid out. The following four visualization functions can be used for this purpose:

- `visualize_graph_2d()` — generates a static 2D graph visualisation using Matplotlib and NetworkX. Node size and color reflect vertex degree. For weighted graphs, edge color reflects weight magnitude using the Plasma colormap. Node labels are shown automatically for graphs with up to 100 vertices. The figure can be saved to disk via `save_as`, making it suitable for inclusion in the paper.
- `visualize_graph_3d()` — generates a static 3D graph visualisation using Matplotlib. The x and y coordinates use the 2D spring layout; the z coordinate uses the Fiedler vector (second eigenvector of the graph Laplacian), giving the third dimension structural meaning — vertices that are spectrally similar are placed at similar heights.

- `visualize_graph_2d_plotly()` — generates an interactive 2D graph visualisation using Plotly. Fully zoomable and pannable, with hover tooltips showing vertex index and degree. Recommended for presentations and exploratory analysis.
- `visualize_graph_3d_plotly()` — generates a fully rotatable interactive 3D graph visualisation using Plotly. The z coordinate uses the Fiedler vector for structural meaning. Most informative for presentations involving graphs with clear community structure. For very large graphs such as the US Power Grid ($n = 4{,}941$), the 2D interactive version is preferred due to rendering cost.

It should be noted that for large graphs, visualization functions can be time-consuming. During our research, we also observed that based on the size of the graph, certain parameters that control the size of the nodes, color scale of the overall visuals, color of the edges, etc need to be modified to enhance visibility and interpretation of the visualisation. Also, certain network repositories, such as networkrepository.com, offer built-in visualization tools that can be used to get a quick visual sense of the data without having to build the visuals from scratch.

4.2. **Dataset: Oregon Router Network.** The Oregon Router dataset, sourced from the SNAP repository, represents a snapshot of Internet autonomous system (AS) peering relationships collected from Oregon route-views data. Each vertex corresponds to an autonomous system, and each edge represents a peering connection between two autonomous systems. After remapping to contiguous vertex IDs (Phase 0), the graph consists of $n = 11{,}174$ vertices and 23,409 edges, with degrees ranging from 1 to 2,389 – a wide spread, as shown in Figures **??** and **??**.

4.3. **Phase 1: Building the Graph Matrices.** The goal of Phase 1 is to map data into workable linear algebra format, where we form three matrices: adjacency matrix, degree matrix and graph Laplacian matrices. These matrices are denoted as A, D and L respectively going forward in this document.

The adjacency matrix, A, captures the relationship between edges and vertices of the graph and in case of unweighted graphs, has zeros and ones to indicate the absence or presence of edges between vertices and for weighted graphs, has numbers other than one to indicate the weights. For undirected graphs, A is symmetric so, $a_{ij} = a_{ji}$ for all i, j. The degree matrix, D, is a diagonal matrix that captures the number of edges incident on every node, with diagonal entries $d_i = \sum_j a_{ij}$ for each j (row sums of A). The Graph-Laplacian matrix, L, is one of the most crucial components of this research, which is computed by $L = D - A$.

The graph Laplacian is central to this project because for any vector v belonging to $\mathbb{R}^n$, the quadratic form satisfies $v^T Lv = \sum_{(i,j)\in E}(v_i - v_j)^2$, showing that L measures the total squared variation of v across all edges. A linear system involving L produces solutions that vary smoothly across connected vertices, making L the natural operator for enforcing spatial correlation in Phase 3. A key property that L holds for a connected graph is that $L\mathbf{1} = 0$. Where $\mathbf{1}$ is a vector of all 1's of the appropriate length. This follows because $(D\mathbf{1})_i = d_i$ and $(A\mathbf{1})_i = \sum_j a_{ij} = d_i$ for every vertex i, so $D\mathbf{1} = A\mathbf{1}$, and therefore $L\mathbf{1} = (D - A)\mathbf{1} = 0$. This implies that L has a zero eigenvalue, which makes it singular and non-invertible. This motivates modifying L

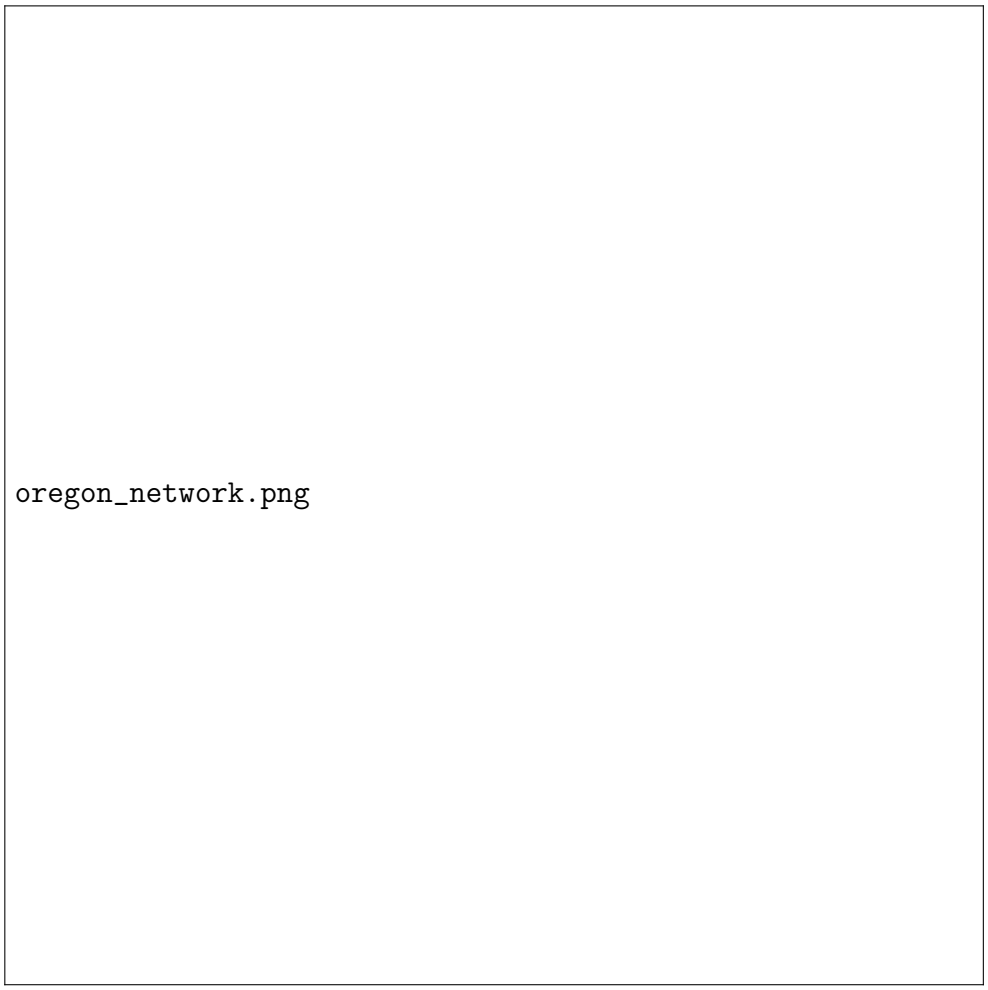


FIGURE 1. Structure of the Oregon Router network (n = 11,174 vertices, 23,409 edges). Node size and color both reflect vertex degree, which ranges from 1 to 2,389.

in Phase 2 and forming the shifted Laplacian which is symmetric positive definite, thus solving the singularity problem.

For relatively smaller graphs, A, D and L can be stored as dense NumPy arrays without running into any significant efficiency issues. However at scale, this becomes largely inefficient — an $11{,}174 \times 11{,}174$ dense matrix requires approximately 953 MB of RAM, whereas the same matrix when stored in sparse CSR format requires only a few megabytes since the vast majority of the entries are zeroes.

In code, this phase is implemented by calling the `build_graph_matrices()` function which takes the input of Phase 0's `load_graph()` function (`edges`, `n_vertices`). It starts by building A first in a List of Lists (LIL) format which facilitates fast incremental construction, and then stores it in CSR format for the downstream arithmetic operations involving A. This flow has been designed such because LIL format is relatively cheaper for construction and CSR is better suited for the operations in the rest of the pipeline. A is used to build D by computing diagonal entries of D as row sums



FIGURE 2. Degree distribution of the Oregon Router network (log scale on the vertical axis). The distribution is heavily right-skewed: most vertices have low degree, with a small number reaching into the hundreds or above.

of A and having the off-diagonal entries as zeroes. L is then computed using $D - A$. Then, all these matrices are efficiently stored as `scipy.sparse.csr_matrix` objects.

The following sanity checks are performed on the output of this function to ensure that desired outputs are obtained:

(1) A is symmetric: $A = A^T$
(2) L is symmetric: $L = L^T$
(3) L has a zero eigenvalue, confirmed by $L\mathbf{1} = 0$
(4) Dimensions of A, D and L are all $n \times n$ where n is the number of nodes.

4.4. **Phase 2: The Shifted Laplacian.** Once the matrices are set up, we move on to phase 2. The goal of Phase 2 is to deal with the singularity of L. This is done by finding the smallest positive eigenvalue of $D^{-1}L$ and utilizing that to compute the shifted Laplacian $\mathcal{L}_\sigma$.

We need L to be invertible so we can solve the system $Lu = f$ in Phase 3 to compute spatially correlated node noises. We make this possible by adding the smallest meaningful value to L while also preserving its original connectivity structure. The shifted Laplacian is constructed using the relation,

$$\mathcal{L}_\sigma = L + \sigma^2 \lambda_{\min} D \tag{4.1}$$

where $\mathcal{L}_\sigma$ is the shifted Laplacian — the invertible version of L. The components are:

- L — the graph Laplacian from Phase 1
- $\sigma^2 = 0.25$ — a hyperparameter controlling the shift magnitude
- $\lambda^+_{\min}$ — the smallest positive eigenvalue of $D^{-1}L$
- D — the degree matrix from Phase 1

Semantically, we do not want to add an arbitrary quantity to L since that would fail to preserve the nature of L and could distort the graph's connectivity structure. This is where $\lambda^+_{\min}$ comes into picture. It is the smallest most meaningful frequency of L — the minimum ratio of variation to connectivity across all non-constant vectors on the graph, characterized by the Rayleigh quotient:

$$\lambda^+_{\min} = \min_{\mathbf{v} \perp \mathbf{1}} \frac{\mathbf{v}^T L \mathbf{v}}{\mathbf{v}^T D \mathbf{v}}.$$

Using $\lambda^+_{\min}$ for the shift ensures the perturbation is calibrated to the graph's own spectral scale rather than being an arbitrary constant. We multiply by D so that the shift is proportional to the degree of each vertex — higher degree vertices receive a larger bump than lower degree vertices, preserving the graph's geometric structure while eliminating the zero eigenvalue. The result $\mathcal{L}_\sigma$ is symmetric positive definite (SPD) — all eigenvalues are strictly positive — and therefore $\mathcal{L}_\sigma$ is invertible.

Implementation. Phase 2 is implemented using two helper functions:

- `compute_lambda_min(L, D)` — computes $\lambda^+_{\min}$ by solving the generalized eigenvalue problem $L\mathbf{v} = \lambda D\mathbf{v}$ and returning the smallest positive result. The generalized form is preferred over computing $D^{-1}L$ directly because it preserves symmetry and is numerically stable. Two implementations are available: `scipy.linalg.eigh(L, D)`, a dense solver that computes all n eigenvalues exactly — accurate but slow for large graphs since it computes eigenvalues we never use; and `scipy.sparse.linalg.eigsh(L, k=2, M=D, which='SM')`, a sparse iterative solver that finds only the 2 smallest eigenvalues. Since the graph is connected, the first eigenvalue is always zero and the second is $\lambda^+_{\min}$. This is significantly faster for large graphs at the cost of a small approximation error controlled by `tol=1e-10`, which is negligible in practice. For the US Power Grid ($n = 4{,}941$), `eigsh` reduces the eigenvalue computation from minutes to seconds. The function returns $\lambda^+_{\min}$ as a float.
- `build_shifted_laplacian(L, D, lambda_min, sigma_squared=0.25)` — takes L, D, and $\lambda^+_{\min}$ as inputs and computes $\mathcal{L}_\sigma = L + \sigma^2 \lambda^+_{\min} D$. A sanity check confirms that the smallest eigenvalue of $\mathcal{L}_\sigma$ is strictly positive, verifying that it is SPD and invertible. The function returns $\mathcal{L}_\sigma$ as output.

4.5. **Phase 3: White Noise Sampling.** This phase marks the beginning of constructing a spatially correlated edge permeability field for our flow network.

4.5.1. *3a. Draw Edge Noise.* In this step, draw as many random numbers from a standard normal distribution as there are edges in the graph. This can be denoted

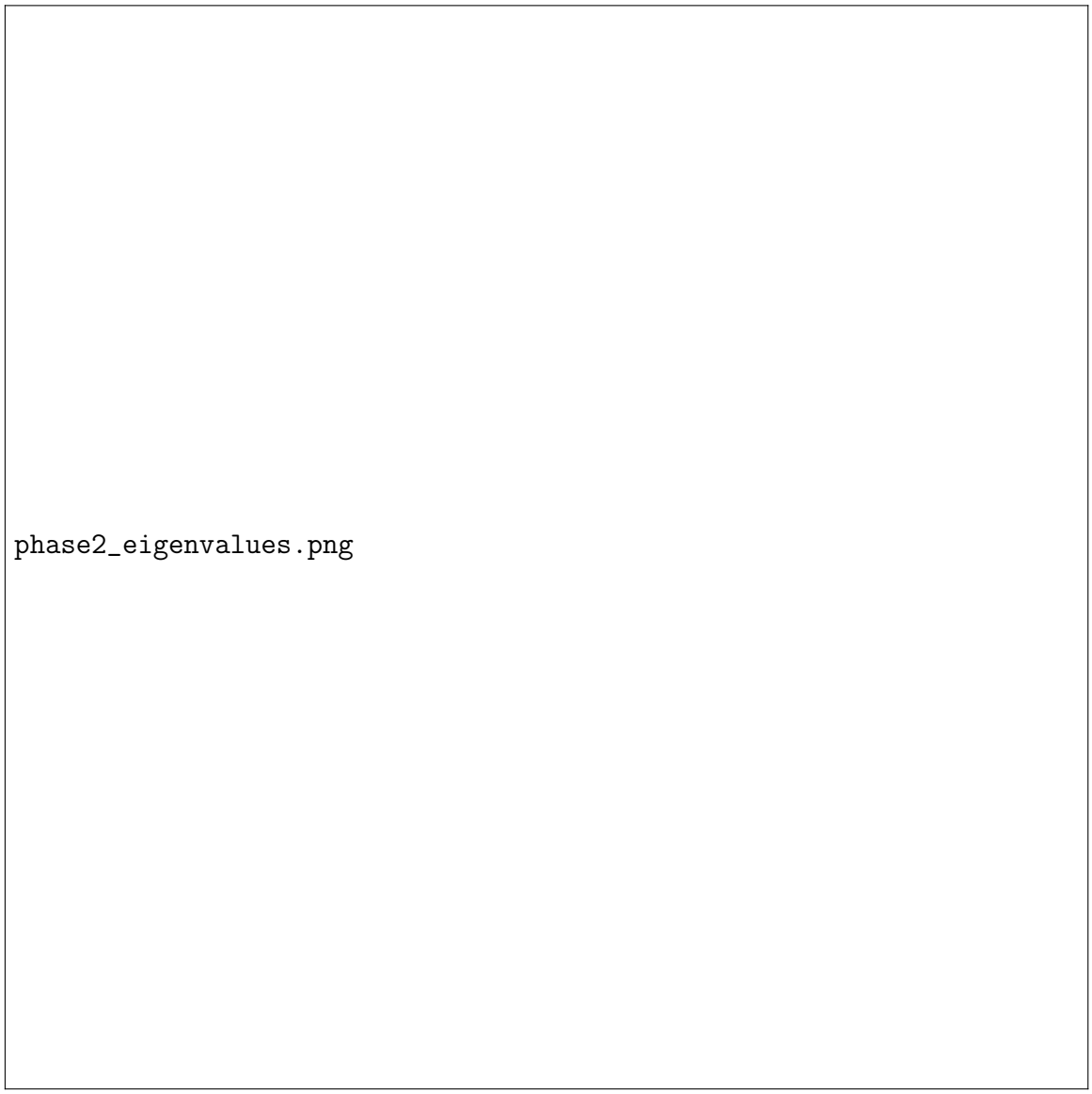


FIGURE 3. Eigenvalues of the generalized problem (L, D) before shifting (left) versus eigenvalues of the shifted Laplacian $\mathcal{L}_\sigma$ after shifting (right), computed on a 15-vertex illustrative graph. The unshifted problem has one eigenvalue at exactly zero, corresponding to graph connectivity; this is what makes L singular. Adding $\sigma^2 \lambda_{\min} D$ shifts every eigenvalue upward by the same relative amount, eliminating the zero eigenvalue and making $\mathcal{L}_\sigma$ symmetric positive definite.

as vector $\mathbf{w}$ such that $\mathbf{w} \in \mathbb{R}^e$ where e is the number of edges. A standard normal distribution is chosen because it produces zero-mean, unit-variance noise — ensuring the random field has no directional bias and that the magnitude of the noise is consistent regardless of the graph size. In code implementation, this is a single call to `numpy.random.randn(len(edges))`.

4.5.2. *3b. Map Noise to Nodes.* In this step, compute f_i for each node i by adding up w_e of each edge e incident on it. In matrix form, this is $\mathbf{f} = B\mathbf{w}$, where $B \in \mathbb{R}^{n\times e}$ for n nodes and e edges. This can be assumed to be a modified version of an incidence matrix where for each edge, its 2 nodes are assigned a value of 1. For larger graphs, $B \in \mathbb{R}^{n\times e}$ can be prohibitively large in dense format — for the Oregon Router Network ($n = 11{,}174$ and $e = 23{,}409$), a dense B requires approximately 2.09 GB of RAM. In code, B is built using the `build_incidence_matrix()` function, which constructs B using LIL format and then converts it to CSR format for further computation. This is the same approach used to build A in Phase 1. The result of this phase is a vector $\mathbf{f}$ such that $\mathbf{f} \in \mathbb{R}^n$ for n nodes. The vector $\mathbf{f}$, however, has values that are uncorrelated. In practice, for the graphs and problems we are considering, neighboring nodes should have similar permeabilities. So, in the next step, we simulate the spatially correlated permeability field.

4.5.3. *3c. Make Noise Spatially Correlated.* In this step, we solve the system $\mathcal{L}_\sigma \mathbf{u} = \sqrt{\lambda_{\min}}\,\mathbf{f}$ for $\mathbf{u}$. This results in a vector $\mathbf{u}$ such that $\mathbf{u} \in \mathbb{R}^n$ for n nodes. Since $\mathcal{L}_\sigma$ encodes graph connectivity, solving this system for $\mathbf{u}$ forces neighboring vertices to agree, thus producing a smooth $\mathbf{u}$ — a spatially correlated Gaussian random field where adjacent vertices have similar values, simulating the spatial correlation observed in real-world permeability fields.

Once $\mathbf{u}$ has been computed and the system $\mathcal{L}_\sigma \mathbf{u} = \sqrt{\lambda_{\min}}\,\mathbf{f}$ has to be solved, we would need to compute $\mathcal{L}_\sigma^{-1}$. However, at scale, computing the inverse of a matrix is computationally expensive and time consuming. To overcome this, we use Cholesky factorization. For a symmetric positive definite matrix such as $\mathcal{L}_\sigma$, Cholesky factorization decomposes it as $\mathcal{L}_\sigma = CC^T$, where C is a lower triangular matrix. Solving $\mathcal{L}_\sigma \mathbf{u} = \sqrt{\lambda_{\min}}\,\mathbf{f}$ then reduces to two triangular substitutions — a forward solve $C\mathbf{y} = \mathbf{f}$ followed by a backward solve $C^T\mathbf{u} = \mathbf{y}$ — each step costing $O(n^2)$ as compared to the $O(n^3)$ cost of computing the matrix inverse.

The key advantage in the context of this project is that $\mathcal{L}_\sigma$ is fixed across all Monte Carlo sample runs. So, $\mathcal{L}_\sigma$ is factorized only once, and the factorized matrices are re-used in each run for two relatively cheaper solves.

Since Phase 3 has three sub-steps, a single function call to `run_phase3()` runs the entire phase in one go. It takes the outputs of Phase 0 (`edges`, `n_vertices`) and Phase 2 (`L_sigma` and `lambda_min`) as input, and returns `u`, `f`, `B` and `factor`.

Sanity Check. To verify that Step 3c produces a smoother field than $\mathbf{f}$, we compare the difference between neighboring vertex values before and after the solve. For each edge $e = (i, j)$, we expect $|u_i - u_j| \ll |f_i - f_j|$. This is because $\mathbf{f}$ consists of sums of independent random numbers — neighboring vertices can have very different f values since there is no constraint forcing them to agree. In contrast, $\mathbf{u}$ is the solution of a linear system involving $\mathcal{L}_\sigma$, which penalizes large differences between connected vertices. As a result, $\mathbf{u}$ exhibits significantly more uniform values across neighbors, directly reflecting the spatial correlation that $\mathcal{L}_\sigma$ enforces.

To visualize the spatial correlation introduced by the Phase 3c solve, we compare the value at each endpoint of every edge in the graph. Figure **??** plots f_i against f_j (left) and u_i against u_j (right) for every edge in the Oregon Router network.

If neighboring vertices' values were independent, no relationship would be visible in either scatter plot. Instead, the raw noise f shows negligible correlation between edge endpoints (Pearson correlation 0.111), while the solved field u shows a strong, visually clear correlation (0.616) – confirming that the Phase 3c solve produces genuine spatial correlation, with connected vertices tending to share similar values.

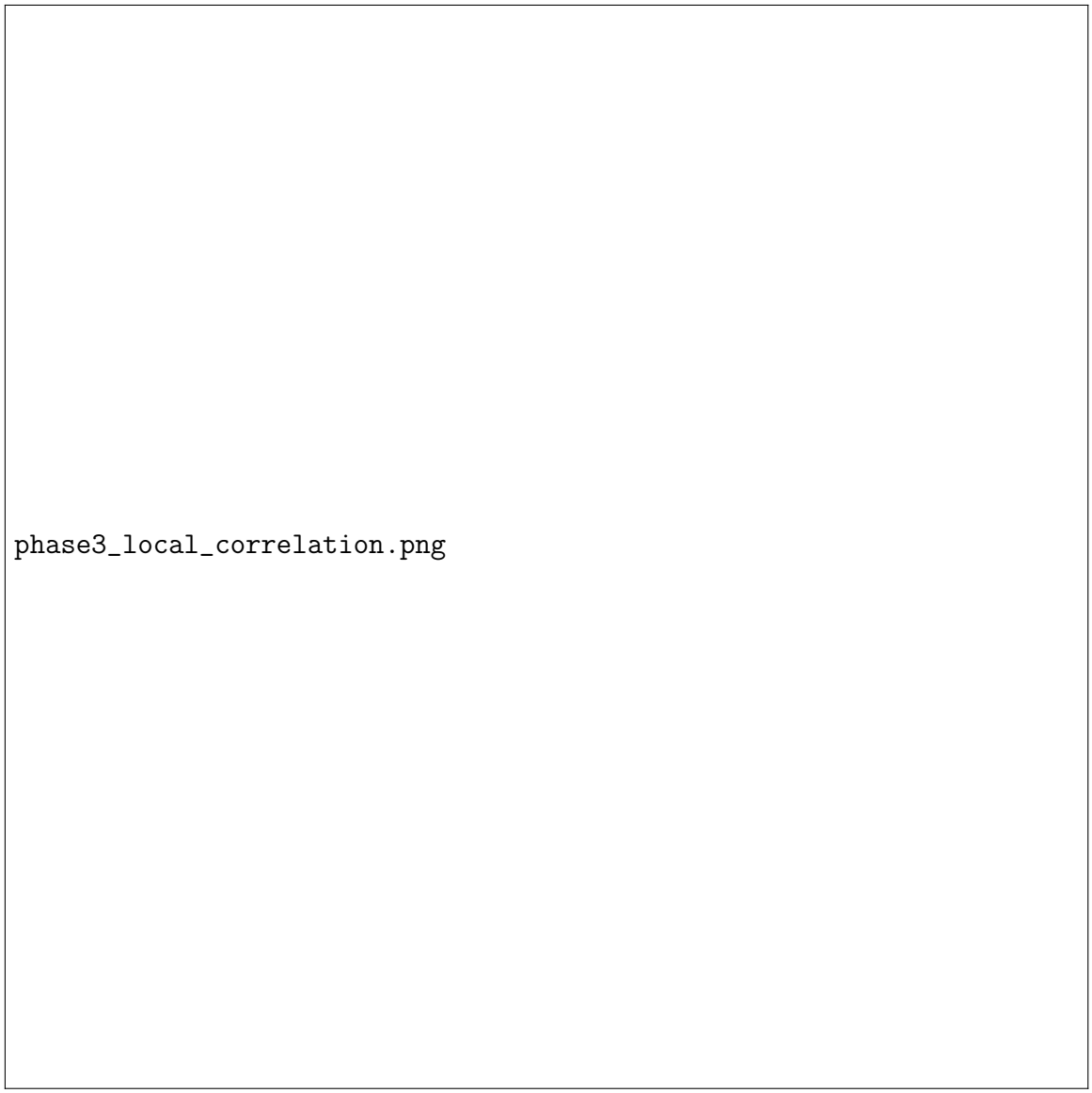


FIGURE 4. Values at each edge's two endpoints, plotted against one another, for the raw noise f (left) and the spatially correlated field u (right), computed on the Oregon Router network ($n = 11{,}174$). Points clustering near the $y = x$ line indicate that connected vertices tend to share similar values. The stark contrast in correlation (0.111 versus 0.616) confirms that the Phase 3c solve introduces genuine spatial correlation.

4.6. **Phase 4: Edge Permeability.** In Phase 4, we compute edge permeabilities for each edge based on the spatially correlated noise computed in Phase 3. For each edge, the following is computed,

$$k_e = \exp\left(\frac{u_i + u_j}{2}\right) \tag{4.2}$$

The rationale behind this relation is that in a simple graph, each edge touches two nodes, and the exp function ensures that we get positive values only, since permeabilities cannot be negative. The result of this phase is a vector $\mathbf{k}$ such that $\mathbf{k} \in \mathbb{R}^e$ where e is the number of edges.

Implementation. In code, it is implemented using a function called `compute_permeability()` which takes two inputs: `u` (from Phase 3) and `edges` (from Phase 0).

For efficiency, the function is fully vectorized using NumPy array indexing — all edge permeabilities are computed simultaneously in a single array operation rather than looping over edges one at a time:

$$\texttt{k_vals} = \exp\left(\frac{\mathbf{u}[\texttt{i_arr}] + \mathbf{u}[\texttt{j_arr}]}{2}\right),$$

where `i_arr` and `j_arr` are precomputed integer arrays of the first and second endpoints of every edge respectively. This vectorization reduces Phase 4 computation time from $\mathcal{O}(|E|)$ Python iterations to a single NumPy call, which is particularly important for large graphs such as the Oregon Router Network ($|E| = 23{,}409$).

A sanity check confirms that all $k_e > 0$ — if this assertion fails it indicates a numerical issue upstream in Phase 3. After Phase 4, every edge in the graph has a permeability weight k_e, forming the weighted graph that is passed to the Darcy flow solve in Phase 5.

4.7. **Phase 5: Weighted Laplacian and Darcy Flow.** Phase 5 focuses on two important tasks: build the weighted Laplacian matrix (L_k) and use that to solve the Darcy flow problem to obtain the pressure field $\mathbf{p}$.

4.7.1. *5a. Build Weighted Laplacian.* In Phase 1, we built an unweighted graph Laplacian L where each edge contributed an equal weight of 1. Later in Phase 4, we computed a random spatially correlated edge weight for each edge. To capture this, we build a weighted Laplacian, denoted as L_k, computed as:

$$L_k = \sum_{e=(i,j)\in E} k_e \, \mathbf{b}_e \mathbf{b}_e^T \tag{4.3}$$

L_k has a similar structure as L, but each edge contributes proportional to its edge permeability value (k_e). High permeability edges dominate the flow, whereas low permeability edges act as bottlenecks. Concretely, for an edge $e = (i, j)$ with permeability k_e, the incidence vector $\mathbf{b}_e$ produces a rank-one contribution to four entries of L_k:

$$L_k[i,i] \mathrel{+}= k_e, \quad L_k[j,j] \mathrel{+}= k_e, \quad L_k[i,j] \mathrel{-}= k_e, \quad L_k[j,i] \mathrel{-}= k_e. \tag{4.4}$$

Summing this contribution over all edges $e \in E$ yields the full weighted Laplacian $L_k = D_k - A_k$, where $A_k[i,j] = k_e$ is the weighted adjacency matrix and $D_k[i,i] = \sum_{e \ni i} k_e$ is the weighted degree.

Implementation. L_k is built using `build_weighted_laplacian(edges, n_vertices, k)`, which takes the edge list and permeability dictionary from Phase 4 as inputs and returns L_k as a sparse CSR matrix. For efficiency, L_k is constructed in COO (Coordinate) sparse format rather than LIL format — COO format appends row indices, column indices, and data values to three arrays simultaneously, which is significantly faster than element-wise insertion for large edge sets.

Because every edge contributes to exactly four entries of L_k (two diagonal, two off-diagonal, as shown above), we construct the COO representation by concatenating four aligned arrays — one entry per edge, per contribution:

```
row_idx = concat(i_arr, j_arr, i_arr, j_arr)
col_idx = concat(i_arr, j_arr, j_arr, i_arr)
```

where `i_arr` and `j_arr` are the edge endpoint arrays. The first two blocks of `row_idx`/`col_idx` address the diagonal entries (i,i) and (j,j); the last two address the off-diagonal entries (i,j) and (j,i). The corresponding data array is built the same way, with $+k_e$ for the diagonal blocks and $-k_e$ for the off-diagonal blocks:

```
data = concat(k_vals, k_vals, -k_vals, -k_vals)
L_k = coo_matrix((data, (row_idx, col_idx)),
                 shape=(n_vertices, n_vertices)).tocsr()
```

The COO constructor automatically sums contributions that map to the same (row, col) position, which correctly accumulates the diagonal entries for vertices with multiple incident edges.

The sparsity structure of L_k (`row_idx` and `col_idx`) is identical across all Monte Carlo steps since the graph topology does not change — only the permeability values k_e change each step. This structure is therefore precomputed once before the Monte Carlo loop and reused at every step; only the `data` array (derived from the current sample's k_e values) is rebuilt each iteration, reducing Phase 5a to a single array construction call per sample rather than a full re-derivation of the sparsity pattern.

A sanity check on L_k verifies three properties:

(1) L_k is symmetric
(2) All $k_e > 0$
(3) All row sums of L_k are zero

confirming that L_k has the same structural properties as the unweighted Laplacian L.

4.8. **Gamma_in / Gamma_out Selection.** Choosing which nodes act as the source set Γ_{in} and sink set Γ_{out} is not governed by a single fixed rule — the appropriate method depends on the graph's underlying topology. Across the four datasets studied, three distinct selection methods were used, each chosen to match the structural characteristics of the corresponding graph rather than applying one approach uniformly.

Approach	**When it fits**	**Used for**
Degree-based	Clear hub-and-spoke structure	Rhesus Brain
Diameter endpoints	No dominant hub, real geometric extent	US Power Grid, Oregon Router
Community detection	Strong modular structure	bio-CE-GN

TABLE 1. Gamma-selection method used per dataset, chosen to match each graph's dominant structural property.

4.8.1. *Approach 1: Degree-Based Selection.* For graphs exhibiting a clear hub-and-spoke topology — a small number of high-degree hub vertices alongside a large population of low-degree peripheral vertices — the source and sink sets are defined directly from the degree distribution:

$$\Gamma_{\text{in}} = \{v : \deg(v) \leq D_{10}\}, \qquad \Gamma_{\text{out}} = \{v : \deg(v) \geq D_{90}\}, \tag{4.5}$$

where D_{10} and D_{90} denote the 10th and 90th percentile of the degree distribution, respectively. This percentile-based cutoff was chosen over an arbitrary fixed count of vertices, since it provides a reproducible, self-documenting rule rather than an unmotivated choice of k. This method was applied to the Rhesus Brain dataset, whose degree distribution shows a clear separation between a small set of hub regions and many peripheral regions — directing flow from the periphery toward the hubs mirrors the graph's natural connective structure.

4.8.2. *Approach 2: Diameter Endpoints.* For graphs without a dominant hub structure but with meaningful geometric or topological extent — such as physical infrastructure networks — Γ_{in} and Γ_{out} are instead defined as neighborhoods around the two most topologically distant vertices in the graph. Computing the exact graph diameter is computationally prohibitive for large graphs, so the endpoints are approximated via a double breadth-first search: from a starting vertex, BFS ***version 1*** identifies the farthest vertex v; a second BFS from v identifies the farthest vertex w. This is repeated from several starting vertices, retaining the pair with the greatest shortest-path distance. The source and sink sets are then taken as the k-hop neighborhoods around v and w:

$$\Gamma_{\text{in}} = N_k(v), \qquad \Gamma_{\text{out}} = N_k(w), \tag{4.6}$$

with any overlap between the two sets removed. This method was applied to the US Power Grid and Oregon Router datasets. We note that $|\Gamma_{\text{in}}|$ and $|\Gamma_{\text{out}}|$ can differ substantially even for identical k, since neighborhood size depends on local vertex density rather than distance alone; for Oregon Router, $k = 4$ yielded $|\Gamma_{\text{in}}| = 11$ and $|\Gamma_{\text{out}}| = 92$. This asymmetry reflects genuine local structure in the graph and is corrected for at the comparison stage via normalization of Q (see Section **??**).

This approach towards gamma selection employs a double-sweep heuristic. The goal is to quantify flow in some sense from one point (could be a group of nodes for large graphs) to another so, we use this method to find two points that are farthest from each other in terms of number of hops. This heuristic proposes a breadth-first search (BFS)***version 2*** from an arbitrary starting vertex, which finds a fathest reachable vertex, and a second BFS from that vertex finds the true opposite diameter

endpoint. This is a simplified variant of the double sweep lower-bound technique described by Crescenzi et al. (**?**), who additionally propose a more sophisticated "fringe" algorithm to close the gap between this lower bound and the graph's true diameter – a refinement not used in this work, where the double-sweep result alone is sufficient to identify two well-separated boundary regions.

{Double-sweep heuristic.} Approximating a graph's exact diameter requires computing the shortest paths between all pairs of nodes, which is obviously expensive for large graphs. ***version 3*** Instead, we use the double-sweep heuristic (**?**) : from an arbitrary starting node s, a breadth-first search (BFS) finds the farthest reachable vertex v ; a second BFS, starting from v, finds the farthest vertex from v, call it w. The distance between v and w is estimated to be the diameter of the graph. The first BFS serves only to locate a plausible extreme point of the graph – since s is an arbitrarily chosen node, it may lie anywhere relative to the graph's true diameter and so the distance between s and v would not be a reliable estimate for the graph's diameter. This is where the second BFS comes in. It starts from v and finds the node that is farthest from v, which we can call w. The distance between v and w can serve as a relatively more reliable estimate to be the graph's diameter since we are measuring distance between two nodes that we consider to be relatively extreme. In general $d(v, w) \geq d(s, v)$, since v was itself the farthest point found from s; running the double sweep from several different arbitrary starting vertices and retaining the largest $d(v, w)$ found further improves the estimate.
Implementation.
Implementation. In code, the double sweep is repeated from several different arbitrary starting vertices, keeping the pair achieving the largest distance $d(v, w)$ found across all attempts, since no single arbitrary starting vertex is guaranteed to yield the true diameter endpoints.

4.8.3. *Approach 3: Community Detection.* For graphs exhibiting strong modular structure — distinct, densely interconnected clusters rather than a hub-spoke or purely geometric topology — Γ_{in} and Γ_{out} are defined using community detection. Communities are identified via greedy modularity maximization (**?**), which begins with each vertex as its own community and iteratively merges the pair of communities yielding the greatest increase in modularity,

$$Q_{\text{mod}} = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j), \tag{4.7}$$

until no further merge improves the score. Clauset, Newman, and Moore note that community sizes found by this algorithm often follow a power-law distribution — a small number of large, dominant communities alongside many small ones — and observe this pattern directly in their own evaluation on a large-scale copurchasing network (**?**). We observe the same pattern across every dataset to which we applied community-based gamma selection: a small number of communities consistently account for the large majority of all vertices, with a long tail of much smaller communities. This is not an artifact specific to our datasets, but a documented, expected

property of greedy modularity maximization, with direct consequences for the boundary fraction precondition discussed in Section **??**.

The source and sink sets are then taken to be the two *largest* detected communities:

$$\Gamma_{\text{in}} = C_{(1)}, \qquad \Gamma_{\text{out}} = C_{(2)}, \tag{4.8}$$

where $C_{(1)}$ and $C_{(2)}$ denote the largest and second-largest communities by vertex count. An earlier variant of this approach instead selected the community pair maximizing average inter-community shortest-path distance; this consistently favored small, structurally unrepresentative fragments over the graph's genuinely dominant modules; for example, an 18- and 15-vertex community pair was selected over the 816- and 736-vertex dominant communities, since small peripheral fragments tend to be more distant in hop-count without reflecting meaningful structure. Selecting the two largest communities directly avoids this failure mode. This method was applied to the bio-CE-GN dataset, whose community structure is dominated by two large clusters (816 and 736 vertices) with a long tail of smaller communities — directly illustrating the power-law-like size distribution described above. ***after the dash sounds VERY AI to me***

We note that community-based selection can yield substantially larger Γ_{in} and Γ_{out} sets than the degree- or diameter-based approaches — hundreds of vertices, as opposed to a handful. Since Q **??** is computed as a sum over edges incident to Γ_{out}, raw Q values are not directly comparable across gamma-selection methods without normalizing by outlet edge count.

4.8.4. *Step 5b: Solve Darcy Flow.* We then use the weighted graph Laplacian to solve the Darcy flow problem. The weighted Laplacian system encodes Darcy's law on the graph — for each edge $e = (i, j)$, the flow is $k_e|p_i - p_j|$, the discrete analogue of Darcy's law in porous media: flux equals permeability times pressure gradient. We prescribe boundary conditions on two disjoint sets of vertices:

- Γ_{in} — inlet vertices with prescribed pressure $p = 1$ (signal source)
- Γ_{out} — outlet vertices with prescribed pressure $p = 0$ (signal sink)

The interior vertices $V \setminus (\Gamma_{in} \cup \Gamma_{out})$ are solved via the reduced system:

$$L_k \mathbf{p} = \mathbf{b}, \tag{4.9}$$

where $\mathbf{b}$ encodes the boundary conditions. The solution $\mathbf{p} = (p_v)_{v \in V}$ is the pressure at every vertex. We verify that all interior pressure values satisfy $0 \leq p_v \leq 1$, confirming physically consistent flow behavior.

Implementation. The solver function used is:

- `solve_darcy(L_k, n_vertices, gamma_in, gamma_out)` — uses `scipy.sparse.linalg.spsolve`, a sparse direct solver. Recommended for graphs up to approximately 10,000 nodes. The boundary and interior vertex partitions are precomputed once before the Monte Carlo loop and reused at every step to avoid redundant computation.

Since L_k changes at every Monte Carlo step because k changes, a fresh solve is performed each time — unlike $\mathcal{L}_\sigma$ which is prefactorized once in Phase 3. For the datasets

tested in this project, `spsolve` with `use_amg=False` was found to be faster than AMG due to the overhead of building the multigrid hierarchy at each step.

4.9. **Phase 6: Quantity of Interest.** The goal of Phase 6 is to extract a single scalar quantity Q that summarizes the total flow through the network for a given Monte Carlo sample.

As per Darcy's law, flow is proportional to the product of permeability and pressure difference. In our case, the flow along each edge $e = (i, j)$ is:

$$\text{flow}_{ij} = k_e \times |p_i - p_j|. \tag{4.10}$$

The absolute value ensures the flow contribution is always non-negative regardless of edge orientation — without it, the pressure difference at outlet vertices would always be negative since $p_{\Gamma_{out}} = 0$ and neighboring interior vertices have $p > 0$, leading to a negative Q which has no physical meaning.

The quantity of interest Q is then the total flux exiting through Γ_{out}, computed by summing the flow contributions of all edges that are incident on at least one outlet vertex:

$$Q = \sum_{\substack{e=(i,j) \\ e \cap \Gamma_{out} \neq \emptyset}} k_e \, |p_i - p_j|. \tag{4.11}$$

By conservation of flow, Q equals the total flux entering through Γ_{in} — what flows in must flow out. This can be verified as a sanity check by computing the inlet flux separately and confirming the two values are equal up to numerical tolerance.

For the Rhesus Macaque Brain Network, Q measures the total neural signal flux reaching the central hub regions from the peripheral sensory regions under one realization of the random permeability field. Running this across N Monte Carlo samples produces a distribution of Q that characterizes uncertainty in signal propagation due to unknown synaptic connection strengths.

Implementation. Phase 6 is implemented in `extract_qoi(p, k, edges, gamma_out)`, which takes the pressure field $\mathbf{p}$ from Phase 5, the permeability dictionary k from Phase 4, the edge list, and the outlet vertex set as inputs, and returns Q as a float. For efficiency, the function is fully vectorized using NumPy masking — an outlet mask identifies all edges incident on Γ_{out} in a single array operation, and the flux sum is computed using `numpy.sum` rather than a Python loop over edges. This is particularly important for large graphs such as the Oregon Router Network where looping over 23,409 edges 1,000 times would add significant overhead.

4.10. **Phase 7: Monte Carlo Simulation.** This phase is all about running N Monte Carlo simulations. We run Phases 3–6 in a loop to generate a random spatially correlated permeability field, solve for its Darcy flow, and extract Q for each sample, then build a distribution to extract the mean and variance.

At each iteration $i = 1, \ldots, N$:

(1) Draw fresh edge noise $\mathbf{w} \sim \mathcal{N}(0, 1)^{|E|}$ (Phase 3a)
(2) Compute $\mathbf{f} = B\mathbf{w}$ and solve $\mathcal{L}_\sigma \mathbf{u} = \sqrt{\lambda_{\min}^+}\, \mathbf{f}$ for $\mathbf{u}$ (Phases 3b, 3c)
(3) Compute edge permeabilities $k_e = \exp((u_i + u_j)/2)$ (Phase 4)
(4) Build L_k and solve $L_k \mathbf{p} = \mathbf{b}$ for pressure field $\mathbf{p}$ (Phase 5)

(5) Extract $Q_i = \sum k_e |p_i - p_j|$ over outlet edges (Phase 6)

This produces an empirical distribution of Q that characterizes uncertainty in flow through the network due to unknown connection strengths.

The expected value of Q is estimated as:

$$\mathbb{E}[Q] \approx \frac{1}{N} \sum_{i=1}^{N} Q_i, \tag{4.12}$$

with standard error $\sigma_Q/\sqrt{N}$. By the strong law of large numbers:

$$\frac{1}{N} \sum_{i=1}^{N} Q_i \xrightarrow{a.s.} \mathbb{E}[Q] \quad \text{as } N \to \infty, \tag{4.13}$$

and the standard Monte Carlo error bound gives:

$$\left| \frac{1}{N} \sum_{i=1}^{N} Q_i - \mathbb{E}[Q] \right| = \mathcal{O}\left(\frac{\sigma_Q}{\sqrt{N}} \right), \tag{4.14}$$

where σ_Q is the standard deviation of Q. This $\mathcal{O}(N^{-1/2})$ convergence rate is independent of the problem dimension — a key advantage of Monte Carlo over deterministic quadrature methods, which suffer from the curse of dimensionality.

Implementation. The Monte Carlo loop is implemented in `monte_carlo_loop_tqdm(L_sigma, lambda_min, edges, n_vertices, gamma_in, gamma_out, edge_weights, N, use_amg)`, which takes $\mathcal{L}_\sigma$ and $\lambda_{\min}^{+}$ from Phase 2 as inputs and returns an array $Q \in \mathbb{R}^N$ of QoI values. Several optimizations are applied to minimize per-sample computation time:

- The Cholesky factorization of $\mathcal{L}_\sigma$, the incidence matrix B, the edge index arrays, the COO sparsity structure of L_k, and the boundary/interior vertex partition are all precomputed once before the loop and reused at every step
- Phase 4 and Phase 6 are fully vectorized using NumPy array operations, avoiding Python loops over edges
- A `use_amg` flag allows switching between `spsolve` (recommended for $n < 50{,}000$) and AMG preconditioned CG (recommended for $n > 50{,}000$)
- A `tqdm` progress bar displays the current sample count, elapsed time, estimated time remaining, and running mean of Q

Analysis. The results are analysed using `analyze_qoi(Q_samples, save_as)`, which prints summary statistics (mean, standard deviation, variance, min, max) and generates two plots: a histogram of the Q distribution with the mean marked, and a convergence plot of the running mean of Q versus number of samples. The convergence plot is the key diagnostic — a flat running mean confirms the estimator has converged and N samples are sufficient for a reliable estimate.

4.11. **Two-Level Monte Carlo.** While the Monte Carlo loop described above converges reliably, each sample requires a full solve on the fine graph, which can become computationally expensive for large n. Two-level Monte Carlo reduces this cost by introducing a smaller *coarse graph* — built by merging groups of fine vertices into single coarse vertices — and combining a large number of cheap samples on the coarse graph

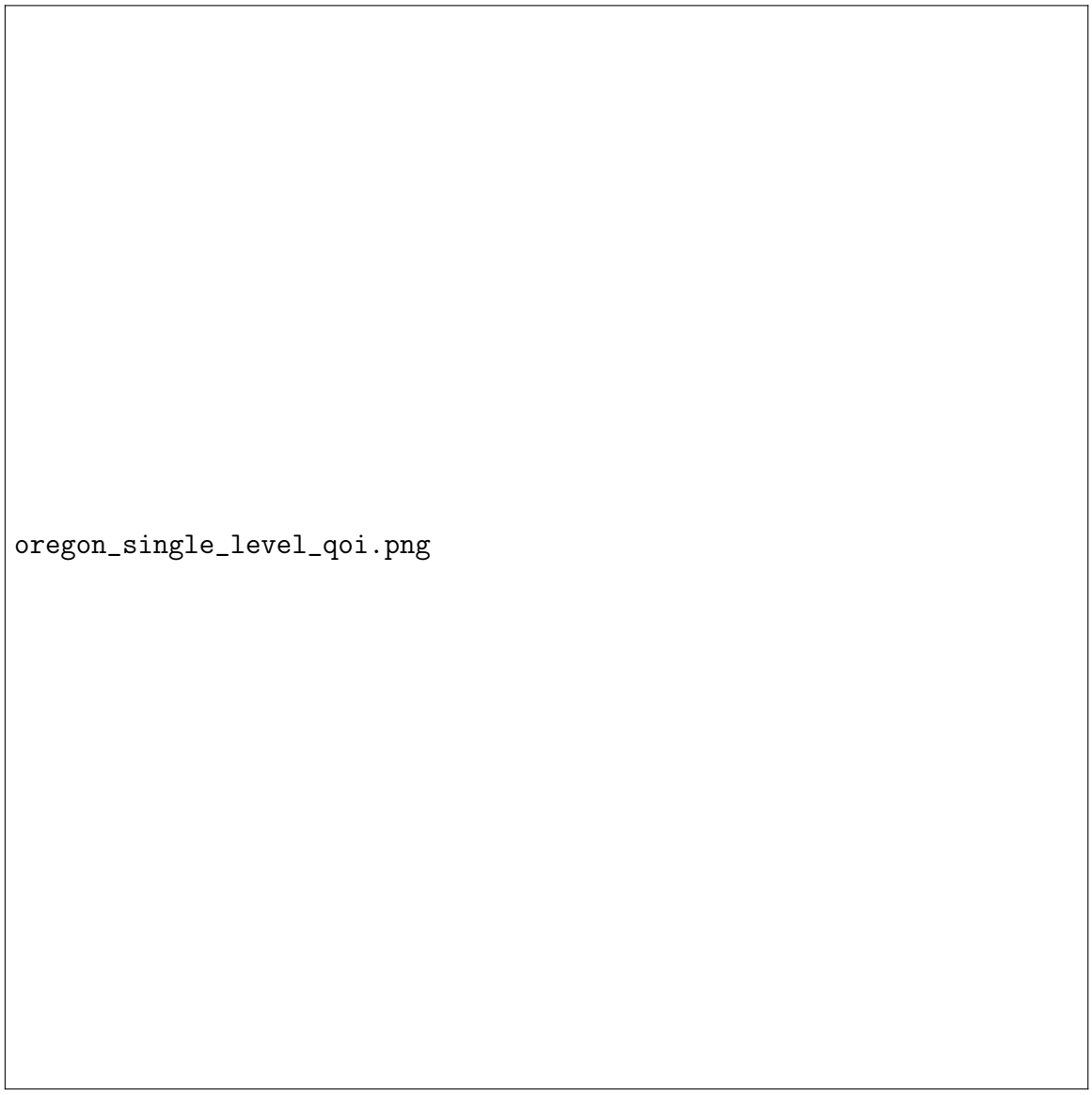


FIGURE 5. Distribution of Q (left) and Monte Carlo convergence (right) for single-level sampling on Oregon Router, $N = 1000$ samples. The running mean stabilizes well before $N = 1000$.

with a smaller number of paired fine/coarse samples used to correct for the coarse approximation's bias. This section describes the aggregation procedure used to build the coarse graph, a diagnostic precondition that determines whether the method is viable for a given dataset, and the results of applying it across six datasets.

4.11.1. *Precondition: Boundary Fraction.* Before attempting aggregation, we check what fraction of the graph is claimed by $\Gamma_{\text{in}} \cup \Gamma_{\text{out}}$:

$$\text{boundary fraction} = \frac{|\Gamma_{\text{in}}| + |\Gamma_{\text{out}}|}{n} \tag{4.15}$$

This is a crucial diagnostic for whether two-level Monte Carlo is likely to succeed. Aggregation will reduce the number of nodes in the non-boundary portion of the graph, regardless of the specific aggregation strategy used (Section **??**) – if Γ_{in} and Γ_{out} together claim a large share of all vertices, the resulting coarse graph can end up with too few (or zero) interior vertices, making the reduced Darcy system in Phase 5b degenerate (no unknowns remain to solve for). We identify a concrete structural mechanism underlying this failure mode on hub-dominated graphs in Section **??**.

Across six datasets tested, we established the following empirical thresholds:

- **Under 1%:** aggregation reliably succeeds (US Power Grid: 0.75%, Oregon Router: 0.92%, bio-grid-yeast with corrected k_{hop}: 0.07%)
- **10% or more:** aggregation fails outright (Rhesus Brain: 25% → zero interior coarse vertices; bio-CE-GN: 70%; bio-grid-yeast via community detection: 76%)

For diameter-endpoint Γ selection specifically, boundary fraction depends on k_{hop} *relative to the graph's diameter*, not on k_{hop} alone. A fixed k_{hop} that works well on a large-diameter graph can fail catastrophically on a small-diameter, hub-dominated graph: $k_{\text{hop}} = 4$ claimed only 0.92% of Oregon Router (diameter 10) but 99.87% of bio-grid-yeast (diameter 5) — the same parameter, applied to a graph whose diameter was too small relative to it. Reducing k_{hop} to 1 on bio-grid-yeast brought the boundary fraction down to 0.07%, restoring a successful outcome.

This raises a natural question: how small should k_{hop} be chosen? In general, k_{hop} should be decreased until the boundary fraction falls comfortably below the empirical safety threshold established above. However, this tuning has a floor: at $k_{\text{hop}} = 0$, the neighborhood expansion is skipped entirely and Γ_{in}, Γ_{out} reduce to the two diameter endpoint vertices themselves. If the boundary fraction remains too large even at this floor value, no further k_{hop} tuning is possible, and the graph itself — rather than the choice of k_{hop} — is the limiting factor.

We verified this directly on a small 15-vertex test graph: even at $k_{\text{hop}} = 0$ (using only the two diameter endpoint vertices as Γ_{in} and Γ_{out}), the boundary fraction was $2/15 \approx 13.3\%$ — already an order of magnitude above the 1% threshold associated with reliable aggregation. This confirms that for sufficiently small graphs, two-level Monte Carlo via diameter-endpoint selection is not viable regardless of parameter tuning: the graph must be large enough that even a single-vertex boundary set on each side remains a small fraction of the total vertex count.

4.11.2. *Aggregation Algorithm.* Once boundary fraction has been checked and found acceptable, the coarse graph is built by grouping fine vertices into aggregates — one coarse vertex per aggregate — with the goal of substantially reducing vertex count while preserving enough of the fine graph's structure that the resulting coarse graph remains a useful approximation for paired sampling (Section **??**).

We considered several existing, documented graph coarsening strategies from the algebraic multigrid and graph partitioning literature, including independent-set-based approaches such as Luby's algorithm. These methods are well suited to coarsening a graph as a single, undifferentiated whole, but our setting imposes an additional structural requirement that general-purpose coarsening does not address: the source

(Γ_{in}), sink (Γ_{out}), and interior vertex roles established during gamma selection must be preserved through coarsening, since the coarse graph inherits the same fixed-pressure boundary conditions used in the fine Darcy solve (Section **??**). A coarse vertex that merged Γ_{in} and Γ_{out} vertices together, for instance, would be required to simultaneously hold pressure 1 and pressure 0 – an ill-posed boundary condition. Consequently, the graph cannot be aggregated as a single undifferentiated unit; the aggregation procedure must be aware of, and respect, the source/sink/interior role of every vertex throughout. This requirement motivated the group-preserving algorithm described below, developed specifically for this constraint rather than adapted from an existing general-purpose method.

Algorithm description. Fine vertices are partitioned into three disjoint groups — Γ_{in}, Γ_{out}, and interior — and aggregated strictly within each group; a vertex is never paired with a neighbor from a different group. Within each group, vertices are processed in order of increasing degree. For each vertex, in turn: if it has an unassigned neighbor within the same group, the two are paired together as a new aggregate; otherwise, if it has a neighbor already belonging to an aggregate that has not exceeded a fixed size cap, the vertex joins that aggregate; otherwise, the vertex is left as a singleton aggregate. Processing vertices by increasing degree gives low-degree vertices, which have the fewest possible neighbors to pair with first opportunity to find a valid partner before their options are claimed by other vertices.

Aggregating strictly within each group guarantees that Γ_{in} and Γ_{out} vertices can never merge into the same aggregate — a coarse vertex cannot simultaneously represent a fixed source ($p = 1$) and a fixed sink ($p = 0$) — without requiring this to be checked as a separate condition. The size cap on aggregate growth prevents a small number of aggregates from absorbing disproportionate edge permeability, which in earlier, uncapped variants of this algorithm inflated Q_{coarse} by 10–100$\times$ relative to Q_{fine} even after normalization.

Coarse graph construction. Regardless of aggregation strategy, every fine edge is checked against the aggregate assignment of its two endpoints. If both endpoints share an aggregate, the edge is internal and dropped; otherwise it contributes to a coarse edge between the two aggregates. A permanent mapping from each coarse edge to the fine edges that contributed to it is retained, since coarse edge permeability must be re-derived from that sample's fine permeability on every Monte Carlo sample, rather than sampled independently.

4.11.3. *Paired Sampling and the Two-Level Estimator.* For a single sample, white noise w is drawn once and used to solve the fine white-noise system (Phase 3) in full, yielding Q_{fine} via Phases 4–6. Coarse edge permeability is then derived, without drawing new randomness, by summing the fine permeability values belonging to each coarse edge according to the fixed mapping described above; the coarse Darcy system is solved identically to the fine one, yielding Q_{coarse}. Because both quantities originate from the same draw of w, they are correlated by construction — this correlation is what keeps $\text{Var}(Q_{\text{fine}} - Q_{\text{coarse}})$ low enough that the correction term can be estimated accurately from a comparatively small number of expensive paired samples, while the



FIGURE 6. Fine graph (left) and coarse graph (right) for Oregon Router, produced by the group-preserving aggregation algorithm. Red vertices are Γ_{in}, cyan vertices are Γ_{out}; gray (fine) and green (coarse) vertices are interior. Node size in the coarse graph reflects aggregate size.

bulk of the sampling budget is spent on relatively cheaper coarse-only samples for the first term of the estimator introduced earlier.

We note that coarse-only sampling is not as inexpensive as it may initially appear: since coarse permeability is derived from the fine field u, every coarse-only sample still requires the full fine white-noise solve (Phase 3) — only the fine Darcy solve (Phase 5b) is actually skipped. On this pipeline, where sparse Cholesky factorization

has already made Phase 3 fast, this means the wall-clock savings from coarse-only sampling are more modest than the variance-reduction figures alone would suggest.

Two diagnostics determine whether the method is providing genuine benefit on a given dataset: the Pearson correlation between Q_{fine} and Q_{coarse} across paired samples, and the variance reduction ratio $\text{Var}(Q_{\text{fine}})/\text{Var}(Q_{\text{fine}} - Q_{\text{coarse}})$. Both are only meaningful once the underlying running means have converged; convergence behavior varies substantially by dataset (see Results), and was checked via running-mean plots rather than assumed from a fixed sample count in every case.

4.11.4. *A Mechanistic Failure Case: Rhesus Brain.* Beyond the boundary-fraction threshold itself, we identified a concrete structural mechanism by which a large boundary fraction causes aggregation to fail on hub-dominated graphs. On Rhesus Brain, using degree-based Γ selection (top-decile degree as Γ_{in}, bottom-decile as Γ_{out}), we found that every one of the 68 interior vertices had *zero* neighbors among other interior vertices — every edge touching an interior vertex connected exclusively to a boundary vertex. This renders the interior block L_{ii} of the weighted Laplacian purely diagonal, decoupling the reduced Darcy system into 68 independent, single-variable equations with no diffusion between neighboring interior vertices. The resulting pressure field collapses to the source value ($p \approx 1$) uniformly across the interior (observed standard deviation 0.0000), rather than showing the smooth gradient expected of a genuine flow solution. This confirms that a large boundary fraction on a hub-dominated graph does not merely reduce the interior vertex *count* — a sufficiently connected hub can render the interior subgraph structurally disconnected from itself, independent of aggregation strategy.

4.12. **Results.** We evaluated the two-level Monte Carlo method across ***six/seven*** seven datasets spanning infrastructure, biological, and social network domains, using the gamma-selection strategy best suited to each dataset's structure (Section **??**).

4.12.1. *Successful Cases.*

Dataset	n	Gamma method	Boundary frac.	Correlation	Var. reduction
US Power Grid	4,941	Diameter-endpoint	0.75%	1.0000	3.11×
Oregon Router	11,174	Diameter-endpoint	0.92%	0.9995	31.25×
bio-grid-yeast	6,008	Diameter-endpoint (k_{hop}=1)	0.07%	1.0000	Strong
Facebook Ego	4,039	Community (top-5% subset)	2.20%	0.9956	12.66×

TABLE 2. Two-level Monte Carlo results on datasets where the method succeeded.

On bio-grid-yeast, the correction was necessary despite modest edge reduction (156,945 fine edges to 145,197 coarse edges, only 7.5% reduction) — a consequence of the graph's high density, which we discuss below. Despite this, correlation and variance reduction both remained strong, since the coarse solve's cost scales with vertex count, which was reduced 2.1×, rather than edge count.

4.12.2. *Failure Cases.*

Dataset	Boundary frac.	Correlation	Outcome
Rhesus Brain	25%	–	Zero interior coarse vertices
bio-CE-GN	70%	–	Predicted failure, not run
C. elegans	3.43%	0.72	Structural checks pass; no statistical benefit

TABLE 3. Datasets where two-level Monte Carlo failed, and why.

4.12.3. *Structural Validity Is Necessary but Not Sufficient.* C. elegans is the most informative failure case, since it passes every structural precondition established above: boundary fraction (3.43%) falls within the range associated with success, aggregation produces a substantial interior (92.5% of coarse vertices), and no $\Gamma_{\text{in}}/\Gamma_{\text{out}}$ overlap occurs. Despite this correlation between Q_{fine} and Q_{coarse} was only 0.72, and variance reduction was effectively 1.05× — no meaningful benefit over direct fine-only sampling.

Comparing C. elegans against Facebook Ego Network — which used the identical gamma-selection strategy (a high-degree subset of the two largest detected communities) and has comparable graph density (mean degree ≈ 48 vs. ≈ 44) — isolates the difference to a single factor: the modularity of the underlying community structure.

Dataset	Modularity	Correlation	Var. reduction
C. elegans	0.26 (weak)	0.72	1.05×
Facebook Ego	0.78 (strong)	0.996	12.66×

TABLE 4. Modularity as a predictor of statistical success, holding gamma-selection method and graph density approximately constant.

We interpret this as follows: modularity measures how genuinely separated a detected community is from the rest of the graph. A low-modularity "community," even when selected via the same greedy algorithm, remains substantially interconnected with the rest of the graph — the resulting $\Gamma_{\text{in}}/\Gamma_{\text{out}}$ do not correspond to a genuinely distinct flow regime, weakening the correspondence between the fine and coarse Darcy solutions even though every structural precondition is satisfied. We note that Clauset, Newman, and Moore's original description of greedy modularity maximization documents a power-law-like community size distribution — a small number of large, dominant communities alongside many smaller ones (**?**) - and we observe the same pattern directly: on bio-CE-GN and bio-grid-yeast, the two largest detected communities account for 70% and 76% of all vertices respectively, which is precisely why whole-community selection was replaced with a high-degree subset in later experiments.

This finding establishes that boundary fraction and interior non-degeneracy, while necessary, are not sufficient preconditions for two-level Monte Carlo to succeed when gamma selection is community-based; the modularity of the selected communities should be checked as an additional diagnostic.

4.12.4. *Comparison to Single-Level Monte Carlo.* To directly quantify the practical benefit of the two-level method, we compare it against single-level (fine-only) Monte

Carlo on Oregon Router, matching both methods to approximately the same target standard error and reporting the number of expensive fine-graph samples required and the wall-clock time for each.

Method	Fine samples	Total samples	Time (s)	SE
Single-level	1000	1000	393.93	0.1092
Two-level (this project)	100	2400	244.78	0.1072
Two-level (general MLMC framework)	100	2400	390.08	0.1086

TABLE 5. Single-level versus two-level Monte Carlo on Oregon Router, matched to approximately the same target standard error. The two-level row is reported both from this project's own implementation and from a general multi-level Monte Carlo framework developed alongside the PDE-based approach this project extends, via the adapter described in Section **??**.

On Oregon Router, two-level Monte Carlo reached approximately the same precision (SE = 0.1072 versus 0.1092) using 10× fewer expensive fine-graph samples (100 paired samples versus 1000 single-level samples), and completed in 244.78 s versus 393.93 s for single-level – a 1.6× wall-clock speedup.

To validate this result independently, we re-ran the same comparison through a general multi-level Monte Carlo framework developed alongside the PDE-based approach this project extends (Section **??**), using an adapter that implements the framework's required interface on top of this project's existing graph pipeline. At the same matched sample counts (2300 coarse-only, 100 paired), the framework reached comparable precision (SE = 0.1086) and a consistent estimate (17.47, versus 17.01 from this project's own implementation and 17.5 from the single-level benchmark), confirming the two implementations agree. The framework's run took 390.08 s, however – noticeably longer than this project's own 244.78 s for the identical sample counts, and longer even than the single-level benchmark's 393.93 s. This overhead likely reflects the framework's additional per-sample bookkeeping (online variance tracking, structured result objects, and residual-norm verification inside its generic linear solver), which this project's own, more minimal implementation does not perform. The statistical benefit of two-level sampling – fewer expensive samples needed for the same precision – therefore transfers correctly across implementations, while the wall-clock benefit is implementation-dependent.

On US Power Grid, at a fixed budget of 300 fine-graph samples, two-level Monte Carlo achieved a modestly better standard error (2.608 versus 2.820) than single-level sampling, but required substantially more wall-clock time (119.47 s versus 26.47 s) – the reverse of the trend observed on Oregon Router. This reflects the same underlying cause discussed above: coarse-only samples on this pipeline still pay the full Phase 3 cost, so the 2000 coarse-only samples used here ($\sim$ 93 s of the total) add substantial time without the corresponding precision benefit seen on Oregon Router, where the coarse Darcy solve's variance reduction was much larger (31.83× versus 3.59× here). Combined with the heavy right-tailed behavior of Q on this dataset (discussed above), US Power Grid illustrates that two-level Monte Carlo's benefit is



FIGURE 7. Convergence of Q_{fine}, Q_{coarse} (left), and the correction term $Q_{\text{fine}} - Q_{\text{coarse}}$ (right) on Oregon Router, over 300 paired samples. Both quantities' running means stabilize within approximately the first 100 samples, visually confirming the high correlation (0.9996) and strong variance reduction (31.83×) reported above.

not universal: its wall-clock advantage depends on both the achieved variance reduction and the dataset's convergence behavior, both of which varied substantially across the datasets tested in this work.

4.13. **Summary.** Across seven datasets, two-level Monte Carlo succeeded reliably when boundary fraction remained under approximately 1% of total vertices, and failed when it exceeded roughly 25%. For diameter-endpoint gamma selection, this threshold is governed by k_{hop} relative to the graph's diameter rather than by k_{hop}

Method	Fine samples	Total samples	Time (s)	SE
Single-level	300	300	26.47	2.820
Two-level	300	2300	119.47	2.608

TABLE 6. Single-level versus two-level Monte Carlo on US Power Grid, both using $N = 300$ fine-graph samples for direct comparability. Unlike the Oregon Router comparison (Table **??**), sample counts here are not matched to a target precision; rather, both methods use the same fixed number of expensive fine-graph samples, and we compare the resulting precision directly.

in isolation, and tuning k_{hop} has a floor below which graph size itself becomes the limiting factor. For community-detection-based gamma selection, boundary fraction alone is insufficient to predict success: the modularity of the detected communities is an additional, necessary diagnostic, as demonstrated by the contrast between C. elegans and Facebook Ego Network. The Rhesus Brain case demonstrates a concrete mechanism by which large boundary fraction causes failure on hub-dominated graphs specifically: sufficiently high-degree boundary vertices can render the interior subgraph disconnected from itself, independent of the aggregation strategy used.

We additionally validated a second aggregation strategy – group-preserving aggregation, which partitions vertices into Γ_{in}, Γ_{out}, and interior and aggregates strictly within each group – against our original boundary-first strategy on US Power Grid and Oregon Router, finding the two strategies effectively interchangeable in terms of correlation and variance reduction. On Oregon Router, we further compared two-level Monte Carlo directly against single-level Monte Carlo at matched precision, finding a 10× reduction in expensive fine-graph samples and a 1.6× wall-clock speedup; the same comparison proved unreliable on US Power Grid and Facebook Ego Network due to the heavy right-tailed distribution of Q, itself a consequence of the log-normal edge permeability model used in Phase 4.

Several research questions raised by this work remain open. First, it is not yet established whether group-preserving aggregation's parity with boundary-first aggregation holds on datasets with large boundary fraction (e.g., Rhesus Brain) or merely on the well-behaved datasets tested so far. Second, the modularity threshold distinguishing successful from unsuccessful community-based gamma selection has only been characterized qualitatively (via a single contrasting pair of datasets) and not established quantitatively across a wider range of modularity values. Third, the heavy-tailed behavior of Q complicates standard-error-based comparisons between sampling methods, and a more robust convergence and comparison methodology – for instance, based on the log-transformed distribution, or on substantially larger benchmark runs – remains to be developed. Fourth, the Minnesota Road Network's convergence instability was never fully diagnosed and remains an open case distinct from the boundary-fraction and modularity failure modes identified elsewhere in this work. Finally, integration with a collaborator's general multi-level Monte Carlo framework demonstrated that this pipeline's two-level structure is the $L = 1$ special case of

a fully general telescoping-sum estimator; whether the boundary-fraction and modularity findings established here extend to a genuine multi-level (three or more level) hierarchy, built by recursively coarsening the coarse graph, has not yet been investigated.

5. Further extensions

Transition to two- and multilevel Monte Carlo simulations (cf., **?**, **??**), and eventually to Markov Chain Monte Carlo (MCMC) and its multilevel extensions (cf., **?**, **?**). Additional planned extensions include:

- **Fractional order sampling:** Extending to fractional powers $\alpha \neq 1$ to control the smoothness of the sampled field, more closely matching the continuous PDE setting.
- **Sparse and multilevel solvers:** Replacing direct solvers with algebraic multigrid (AMG) preconditioned conjugate gradient for scalability to larger graphs beyond the current dense solver limits.

Fariborz Maseeh Department of Mathematics and Statistics, Portland State University, 1855 SW Broadway, Portland, OR 97201, U.S.A.

Email address: dpatel@pdx.edu, panayot@pdx.edu